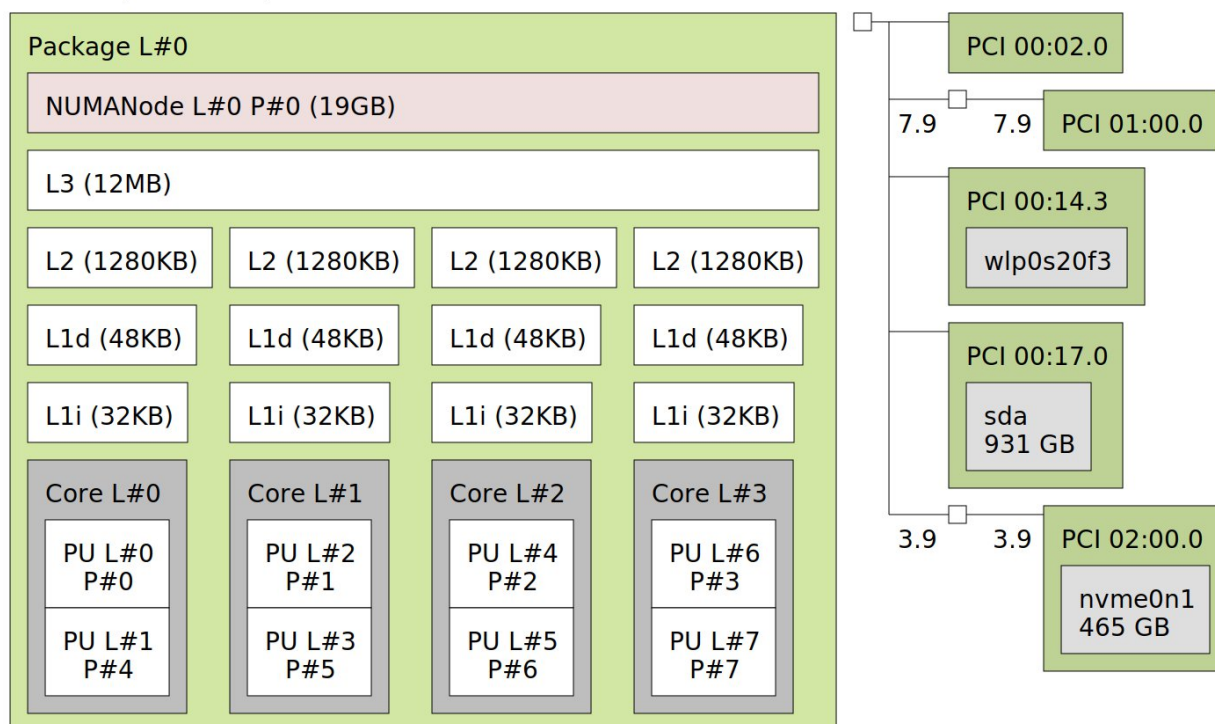


Machine (19GB total)



Host: hanieh-IdeaPad-3

Date: ۱۹:۲۶:۳۰ نئوژ ۰۵ هـ عمّوج

سناریو هایی که می‌خواهیم مورد بررسی قرار دهیم:

- Sequential vs Random Access
- Working Set Size Sweep
- Modification
- Multi thread
- Multi process

سناریو ۱: Sequential Access vs Random Access

هدف این آزمایش این است که ببینیم locality حافظه چه تأثیری روی cache miss دارد.

با اجرای کامند زیر داده های قبلی را پاک میکنیم:

redis-cli FLUSHALL

با یک اسکریپت پایتون یک دیتاست اولیه به سائز ۱۰۰۰۰۰ ایجاد میکنیم.

با یک اسکریپت دیگر به صورت ترتیبی و sequential آنها را میخوانیم.

خروجی perf:

6,267,567,834	cycles:u		
4,792,114,005	instructions:u	#	0.76 insn per cycle
18,986,427	cache-references:u		
9,215,999	cache-misses:u	#	48.540 % of all cache refs
1,328,017,556	L1-dcache-loads		
112,684,980	L1-dcache-load-misses	#	8.49% of all L1-dcache accesses
33.482335118	seconds time elapsed		

یک اسکریپت دیگر برای دسترسی random مینویسیم.

خروجی perf:

5,967,516,001	cycles:u		
4,623,269,858	instructions:u	#	0.77 insn per cycle
13,506,453	cache-references:u		
7,042,194	cache-misses:u	#	52.139 % of all cache refs
1,281,132,014	L1-dcache-loads		
111,867,145	L1-dcache-load-misses	#	8.73% of all L1-dcache accesses
31.582493557	seconds time elapsed		

ممیبنیم که نتایج به شدت مشابه یکدیگر است. حتی با دیتاست بزرگتر هم نتیجه همین است. احتمالا نحوه ی ذخیره سازی دیتا در redis بصورت hash شده بوده که این نتیجه حاصل شده.

سناریوی ۲: working set size

با کامند زیر حافظه ی کش سیستم خودمان را پیدا میکنیم:

lscpu | grep cache

L1d cache:	192 KiB
L1i cache:	128 KiB
L2 cache:	5 MiB
L3 cache:	12 MiB

البته این مشترک بین همه ی هسته ها است و برای سطح ۱ و ۲ باید تقسیم بر ۴ شود.

میخواهیم هر سه سطح کش را بررسی کنیم. اول باید چک کنیم که هر کلید تقریباً چقدر حافظه میگیرد.

ابتدا قبل از اینکه دیتابیس با چیزی پر شود با کامند زیر حافظه ی اشغال شده را پیدا میکنیم:

redis-cli INFO memory | grep used_memory:

used_memory:858528

سپس ۱۰۰۰ کلید اضافه میکنیم و دوباره حافظه را اندازه گیری میکنیم:

used_memory:945920

پس هر entry تقریباً 87.392 بایت فضا اشغال میکند.

100 کلید: زیر سطح ۱

4,012,525,628	cycles:u		
4,680,056,072	instructions:u	# 1.17	insn per cycle
4,788,063	cache-references:u		
696,960	cache-misses:u	# 14.556 %	of all cache refs
1,300,667,233	L1-dcache-loads		
103,194,606	L1-dcache-load-misses	# 7.93%	of all L1-dcache accesses
415,462	LLC-loads		

45,574	LLC-load-misses	#	10.97% of all LL-cache accesses
30.001648073 seconds time elapsed			

۱۰۰۰ کلید: زیر سطح ۲

4,117,528,988	cycles:u		
4,695,925,012	instructions:u	#	1.14 insn per cycle
6,430,226	cache-references:u		
858,761	cache-misses:u	#	13.355 % of all cache refs
1,304,833,768	L1-dcache-loads		
111,556,568	L1-dcache-load-misses	#	8.55% of all L1-dcache accesses
775,639	LLC-loads		
82,567	LLC-load-misses	#	10.65% of all LL-cache accesses
30.002199037 seconds time elapsed			

۱۰۰۰۰ کلید: زیر سطح ۳

4,586,321,332	cycles:u		
4,746,299,067	instructions:u	#	1.03 insn per cycle
9,806,231	cache-references:u		
3,015,835	cache-misses:u	#	30.754 % of all cache refs
1,312,184,974	L1-dcache-loads		
110,820,928	L1-dcache-load-misses	#	8.45% of all L1-dcache accesses

1,771,371	LLC-loads		
590,427	LLC-load-misses	#	33.33% of all LL-cache accesses
30.001904029 seconds time elapsed			

۱۰۰۰۰۰ کلید: بعد سطح ۳

5,786,744,304	cycles:u		
4,703,443,317	instructions:u	#	0.81 insn per cycle
13,341,367	cache-references:u		
7,354,460	cache-misses:u	#	55.125 % of all cache refs
1,295,158,532	L1-dcache-loads		
109,561,378	L1-dcache-load-misses	#	8.46% of all L1-dcache accesses
2,799,538	LLC-loads		
1,925,900	LLC-load-misses	#	68.79% of all LL-cache accesses
30.001595728 seconds time elapsed			

سناریوی ۳: modification

100000 داده در دیتابیس مینویسیم. حالا در یک حلقه همه ی آنها را تغییر میدهم و به جای کاراکتر A کاراکتر B را مینویسیم. خروجی perf:

14,955,219,944	cycles:u		
14,516,511,349	instructions:u	#	0.97 insn per cycle
98,684,934	cache-references:u		
64,880,123	cache-misses:u	#	65.745 % of all cache refs

3,905,683,926	L1-dcache-loads		
352,903,862	L1-dcache-load-misses	#	9.04% of all L1-dcache accesses
9,352,267	LLC-loads		
5,591,925	LLC-load-misses	#	59.79% of all LL-cache accesses
66,691	page-faults:u		
77.027100689 seconds time elapsed			

یکبار دیگر همینکار را میکنیم ولی اینبار سایز داده را هم تغییر میدهیم. بصورت یکی در میان داده ها را کوچکتتر و بزرگتر میکنیم. خروجی:

17,332,383,856	cycles:u		
16,201,257,597	instructions:u	#	0.93 insn per cycle
139,952,514	cache-references:u		
105,568,197	cache-misses:u	#	75.431 % of all cache refs
4,348,731,451	L1-dcache-loads		
423,102,865	L1-dcache-load-misses	#	9.73% of all L1-dcache accesses
9,846,153	LLC-loads		
6,777,865	LLC-load-misses	#	68.84% of all LL-cache accesses
147,663	page-faults:u		
75.601463885 seconds time elapsed			

سناریوی ۴: multi-thread

Redis ذاتا برنامه ی single thread ای است ولی سعی میکنیم بصورت multi thread آنرا اجرا کنیم.

برای 1 thread:

1,432,549,747	cycles:u		
926,592,409	instructions:u	#	0.65 insn per cycle
5,793,836	cache-references:u		
3,121,471	cache-misses:u	#	53.876 % of all cache refs
256,834,464	L1-dcache-loads		
23,632,592	L1-dcache-load-misses	#	9.20% of all L1-dcache accesses
1,123,599	LLC-loads		
812,135	LLC-load-misses	#	72.28% of all LL-cache accesses
14.688842154	seconds time elapsed		

4 threads:

5,978,459,923	cycles:u		
3,694,402,757	instructions:u	#	0.62 insn per cycle
24,693,492	cache-references:u		
10,868,999	cache-misses:u	#	44.016 % of all cache refs
1,023,278,748	L1-dcache-loads		
112,134,016	L1-dcache-load-misses	#	10.96% of all L1-dcache accesses
4,715,952	LLC-loads		

3,016,194	LLC-load-misses	# 63.96% of all LL-cache accesses
34.302431311 seconds time elapsed		

:threads 8

12,954,916,642	cycles:u	
7,395,359,735	instructions:u	# 0.57 insn per cycle
105,820,309	cache-references:u	
17,809,073	cache-misses:u	# 16.830 % of all cache refs
2,048,028,971	L1-dcache-loads	
258,648,201	L1-dcache-load-misses	# 12.63% of all L1-dcache accesses
15,630,504	LLC-loads	
5,226,790	LLC-load-misses	# 33.44% of all LL-cache accesses
71.315440804 seconds time elapsed		

زمان پردازش بیشتر شده پس احتمالاً در ردیس یک bottleneck وجود دارد مثلاً سرور ردیس به همه ی کلاینت ها باید سرویس دهد.

Miss rate کمتر شده.

1,275,019,683	cycles:u		
919,067,764	instructions:u	# 0.72	insn per cycle
4,727,165	cache-references:u		
2,853,900	cache-misses:u	# 60.372	% of all cache refs
255,550,013	L1-dcache-loads		
20,184,481	L1-dcache-load-misses	# 7.90%	of all L1-dcache accesses
987,591	LLC-loads		
749,336	LLC-load-misses	# 75.88%	of all LL-cache accesses
9.132298287	seconds time elapsed		

5,689,786,867	cycles:u		
3,513,855,887	instructions:u	# 0.62	insn per cycle
15,911,123	cache-references:u		
7,798,289	cache-misses:u	# 49.012	% of all cache refs
969,055,471	L1-dcache-loads		
111,991,356	L1-dcache-load-misses	# 11.56%	of all L1-dcache accesses
4,105,160	LLC-loads		
2,721,390	LLC-load-misses	# 66.29%	of all LL-cache accesses

12.652420215 seconds time elapsed

Λ

11,662,790,477	cycles:u		
6,743,692,490	instructions:u	#	0.58 insn per cycle
173,716,981	cache-references:u		
16,167,811	cache-misses:u	#	9.307 % of all cache refs
1,857,726,682	L1-dcache-loads		
206,093,058	L1-dcache-load-misses	#	11.09% of all L1-dcache accesses
30,598,122	LLC-loads		
5,273,538	LLC-load-misses	#	17.23% of all LL-cache accesses
16.442933098	seconds time elapsed		